

考虑预测更新与运行平稳性的微网购电调控策略

摘要

对于问题一,

对于问题二,

对于问题三,

对于问题四,

最后,

关键词： 关键词； 关键词； 关键词； 关键词； 关键词

一、 问题背景与问题重述

1.1 问题背景

分布式光伏为社区提供了就地获取电能的途径，但其电力供给的时间分布与居民用电需求并不天然一致：发电充裕时可能出现剩余电量，而负载集中或光伏回落时又需要依赖外部电网。配置储能电池可以将不同时段电能供需连接起来，却也引入了充放电损耗、功率上限和安全储电范围等限制。因此，微网运营的关键并非单纯提高某一时段的自给率，而是在持续满足负载的前提下，协调购电时机、储能状态与预测风险，控制整个运行周期的购电支出。现实调度还存在“计划先制定、信息后到达”的矛盾。购电计划偏乐观可能引发高价紧急购电，过度保守则可能为未被利用的电量支付费用；中途调整购电安排有助于修正计划，但调整本身也有代价。因此，设计一个“预测—计划—修正—执行—结算”一体化的调度分析模型，对建设兼具经济性与可靠性的微网发电系统具有重要意义。

1.2 问题重述

本文以光伏电源、社区负载、储能电池及外网组成的系统为对象，将原题四问归纳为以下递进任务。

问题一要求利用典型日电价、负载及给定光伏预测，确定各时段的购电与充放电安排。我们建立两层混合整数线性规划模型，在日初日末储电量一致条件下，先求最低购电费，再在小幅费用让步范围内改善电池充放电的平稳性。购电量作为题目要求的结果进行统计，不再单独最小化。

问题二要求先依据历史负载和光伏数据构建一个预测模型，根据预测得到的每日负载和发电数据制定每日 0 时计划。新的购电计划需要考虑如果计划供电低于实际负载，需要按当时电价 5 倍计收的紧急购电费用。

问题三引入每日 0、6、12、18 时发布的未来 24 小时整点光伏预报，研究是否以及如何修正未执行计划。重点衡量提前量误差、未来更新机会与上调 1.5 倍、下调 50% 相关计费之间的关系，并评价额外预报的实际决策价值。

问题四将固定日内价格扩展为随日期和时段变化的电价，在明确价格可见性的前提下分别重新研究无额外光伏预报和有日内预报的两类策略，比较价格预测误差与供需误差共同作用下的购电安排。

二、问题分析

四问共享能量守恒、储能状态递推与充放电硬互斥约束，差别在于决策时可用的信息及购电结算机制。本文按照“物理约束统一、历史信息预测、风险驱动决策、因果执行检验”的思路，逐层增加供需误差、日内预报和电价波动。数据对齐、参数拟合和误差校准均限定于当时已知信息，以保证各策略在同一可行域和可比的信息条件下评价。

问题 1 分析

本问的电价、负载和光伏预测已知，关键在于利用储能连接不同时段的供需，同时兼顾购电费用与充放电平稳性。首先按十分钟区间建立能量平衡和储电递推关系，以二元变量禁止同时充放电，并要求日初日末电量相等。随后结合功率变化和最短运行时间限制，先求当前运行条件下的最低电费，再在费用预算内减少功率波动与短动作。最后按指定时段汇总结果，通过经济基准对比、约束检查和参数扰动，分析平稳运行的代价及计划对预测偏差的适用范围。

问题 2 分析

日前购电必须在实际供需揭晓前确定，而计划剩余与供电不足的费用不对称。先以历史同期曲线和卷积残差网络预测负载、光伏，再由条件误差决策树构造逐槽净需求支持，采用离散储能动态规划权衡计划购电、缺口风险及操作强度。实际运行时锁定购电量，允许电池按当槽余缺改变充放电方向，并连续传递真实储电量。最后分别核算购电费用、吞吐、反转和功率变化，通过同预测输入下的规划及执行对照评价模型。

问题 3 分析

关键矛盾是新预报能降低部分不确定性，但调整有成本，远期风险还可能被后续更新修正。首先按发布时刻与提前量匹配预报和实际值，分离偏差与方差，并控制昼夜差异；再生成保留时间相关性的误差路径，将整点功率校准后插值积分至 10 分钟。进而建立提前量感知的多阶段风险调度，使 0 时计划预先考虑未来更新机会，同一已知历史对应相同动作。预报到达后重新优化未执行区间，统一结算并通过不同更新时刻组合评价信息价值。

问题 4 分析

关键矛盾由供需预测误差扩展为供需与价格的联合不确定性，预知真实全日电价会夸大策略能力。首先以周期、近期及前周同刻价格建立滚动动态回归，随后分别接入问题 2 和问题 3，保持两类策略的光伏预报权限不变。若联合场景导致计算负担过大，再按决策成本压缩场景并回填遗漏的极端路径。最后在相同滚动时序下比较预测、费用与计算开销，区分价格环境变化、信息增加及算法增强各自的影响，全程不提前使用未来实测值。

三、模型假设

各问共同采用以下五项假设，问题一的典型日条件在后面补充说明。容量、功率、安全储电范围和供电要求属于题面约束，不作为可以放宽的假设重复列出。

假设 1：采用集中式单母线、区间平均功率表征。将微网视为统一调度的聚合系统，不另行刻画内部线路潮流、电压与频率动态；10 分钟控制区间内用平均功率描述能量交换。**合理性依据：**附件提供的是聚合功率与电价，没有内部拓扑、线路阻抗及秒级控制数据，不足以识别网络与暂态模型。**模型影响：**可用区间能量平衡连接负载、光伏、储能和购电；结论限于能量管理层，不声称验证配电网电气安全。整点预报通过插值后积分转换，不直接视为小时平均值。

假设 2：总程效率为 90%，且充、放电方向对称。取 $\eta_c = \eta_d = \sqrt{0.9}$ ，主模型中不另计自放电、辅助设备耗电与容量随时间衰减。**合理性依据：**题面只给出一个效率数值，没有分方向效率曲线及老化、辅助用电参数；对称分解是保持总程效率并减少不可识别参数的一种约定，而非由题面唯一推出的物理事实。**模型影响：**得到统一、可核验的储电量递推关系；通过效率及附加损耗情景检验结论敏感性，不将平稳性直接折算为延寿收益。

假设 3：负载不可主动削减，外网正常可用且不设置额外购电上限。所有负载均需得到满足；不设置售电收益，剩余电量允许不被利用，已申报购电仍按适用规则结算。**合理性依据：**题目要求保障供电，给出了紧急购电机制，却未给出可转移负荷、外网容量上限、停电过程及售电价格。**模型影响：**紧急购电承担最后的供需平衡补救，比较聚焦于费用而非失负荷；总剩余应区分光伏弃电与已付费未使用购电。本结论不适用于限电或孤岛场景。

假设 4：信息按时可用，决策与执行遵守因果顺序。已发布预报和已实现观测可在下一次允许决策时使用，不考虑超过 10 分钟的通信或执行延迟；问题 4 主情景暂按真实价格随时间揭示处理。**合理性依据：**题面明确了预报发布时间，但未提供通信故障和价格提前公告机制；逐步揭示价格是避免引入额外先知信息的工作解释。**模型影响：**只用过去信息拟合预测与风险参数，冻结已执行动作并传递实际储电量；价格全日已知仅作辅助情景，延迟影响可通过滞后一期的信息检验评估。

假设 5：经周期与近期状态调整后，历史误差在短期内具有局部可迁移性。不要求全年同分布、误差独立、无偏或服从正态分布，只认为相近时段及条件下的近期误差能提供下一阶段风险参考。**合理性依据：**日历周期可作为建模结构，但其强度与误差分布需随历史更新；在缺乏未来气象等信息时，局部历史是可获取的校准依据。**模型影响：**支持滚动回归、备用分位与相关场景生成；采用近期加权、分组收缩、分块重采样和滚动外推检验，发现漂移时更新窗口，避免将该假设当成已验证事实。

问题一的补充假设

补充假设 1：假设内容：给定光伏预测作为本问的确定性供给输入，外网可按计划提

供所需电量，不额外设置购电容量上限。**设立依据：**本问要求利用已给定预测制定计划，未给出预测误差分布或外网接入功率限制。**对模型的影响：**约束保证给定预测下逐段供电；预测偏离时的适用边界另作压力测试，不据此假定实际发电无误差。

补充假设 2： **假设内容：**将给定曲线视为可重复的典型日，以 6000 kWh 为日初、日末储电量，并循环连接午夜两侧的运行模式。**设立依据：**问题一给出相同的日负载、电价，要求日初日末储电量相等，附录给出初始电量 6000 kWh。**对模型的影响：**排除透支初始储能获得虚假节费的方案；循环运行条件避免把功率跳变和短动作转移到日界处。

其中，典型日循环是本问的运行约定，后续随机变化条件下仍需按真实储电量跨日传递。爬坡限值及最短运行时间作为平稳运行方案的可调设置，在模型中单独列出，不视为题面已给定的硬件参数。

四、符号说明

下表列出贯穿各问的公共符号，局部回归系数和辅助线性化变量在相应模型中定义。上标“ $\sim$ ”表示预测量；日下标 d 在不致混淆时省略。 E 表示储电量， e 表示紧急购电量，两者严格区分。

表 1 主要符号及单位

符号	含义	单位
d, t	日期索引、日内调度区间索引, $t = 1, \dots, 144$	—
Δt	单个调度区间长度, 取 1/6	h
τ, h	预报发布时刻、距离发布时刻的预测提前量	h
$L_{d,t}, V_{d,t}$	负载、可用光伏的区间平均功率	kW
$\hat{L}_{d,t}, \hat{V}_{d,t}$	决策时可获取的负载、光伏预测功率	kW
$p_{d,t}, \hat{p}_{d,t}$	基础电价及其预测值, 交易倍数另计	元/kWh
$g_{d,t}$	每日 0 时申报的计划购电量	kWh
$q_{d,t}$	当前决策版本中调整后的总购电量	kWh
$e_{d,t}$	实际执行时为补足缺口购买的紧急电量	kWh
$a_{d,t}^+, a_{d,t}^-$	相对原计划的净上调量、净下调量	kWh
$P_{d,t}^c, P_{d,t}^d$	交流侧充电功率、放电功率	kW
$z_{d,t}^c, z_{d,t}^d$	充电、放电模式指示变量, 二者不可同时为 1	—
$E_{d,t}$	区间 t 开始时的储电量, $E_{d,145}$ 为日末值	kWh
$E_{\min}, E_{\max}$	安全储电量下限 1200、上限 10800	kWh
$P_{\max}$	最大充电或放电功率, 取 5000	kW
η_c, η_d	充电、放电效率, 均取 $\sqrt{0.9}$	—
$w_{d,t}$	供需平衡中的总未利用电量, 并非全部弃光	kWh
$R_{d,t}$	为应对未来缺口预留的电池内部能量	kWh
$\varepsilon_{\tau,h}$	对应发布时刻与提前量的光伏功率预报误差	kW
$b_{\tau,h}, \sigma_{\tau,h}$	该组误差的偏差、去偏后的标准差	kW
$\mathcal{I}_{\tau}$	决策时刻 τ 已可获取的信息集合	—
s, π_s	随机场景索引及相应概率	—
C_{tot}	计划、调整及紧急购电构成的总费用	元
α, λ	尾部风险置信水平、风险厌恶权重	—

其中, $a_{d,t}^+ = (q_{d,t} - g_{d,t})^+$, $a_{d,t}^- = (g_{d,t} - q_{d,t})^+$ 仅表示相对原计划的净偏差, 不预先确定多次调整的计费基准。预报误差取“实际值减预测值”, 其 MSE 单位为 kW^2 ; $\sigma^2 = \text{MSE} - b^2$ 须在同一分组与一致矩估计口径下使用。模型中的功率需乘以 Δt 才成为区间电量。

五、问题一的模型建立与求解

5.1 问题一模型的建立

本文将充放电功率的平稳协调作为问题一的核心改进，在满足供电需求的同时，关注短时功率剧烈变化与碎片化充放电，避免仅以单日购电费评价调度优劣。为此，将功率总变差、充放电吞吐及方向反转纳入方案评价与选择，在可接受的费用条件下改善电池运行轨迹。

这一设计具有电池老化与经济运行依据。以锂离子电池为例，实验研究表明，高倍率或低温等不利充电条件会增强极化并诱发析锂，造成活性锂损失、容量衰减及寿命缩短 [?]。因此，当剧烈功率变化伴随不利电流、温度及循环条件时，其老化风险不能忽略。循环退化也是储能运行成本的重要组成部分，可能抵消短期电价套利收益，调度需兼顾电费与设备寿命代价 [?]。

不过，碎片化动作的影响还取决于循环深度、荷电状态和休息时段；已有实验发现，部分动态工况反而可延长寿命 [?]。故本文将平稳性视为约束不必要操作的工程指标，不将所有功率波动等同于严重老化，也不在缺少电芯与温度数据时虚构折旧费用。模型以储电量和最近非空充放电方向为状态，采用动态规划协调购电与储能，并保持日初日末储电量一致。

输入与物理约束。输入附件 1 的负载 L_t 、光伏预测 V_t 及电价 p_t ，按 10 分钟划分为 $T = 144$ 段， $\Delta t = 1/6$ h。采用区间终点时标，净需求为 $n_t = (L_t - V_t)\Delta t$ 。记 c_t, d_t 为交流侧充、放电量，则

$$\begin{aligned} g_t + d_t &= n_t + c_t + w_t, & g_t, w_t &\geq 0, \\ E_{t+1} &= E_t + \eta_c c_t - d_t / \eta_d, & \eta_c &= \eta_d = \sqrt{0.9}, \\ 1200 &\leq E_t \leq 10800, & E_1 &= E_{T+1} = 6000. \end{aligned} \tag{1}$$

其中， g_t, w_t 分别为购电量和未利用剩余电量， E_t 为段初储电量，单位均为 kWh。充、放电量不超过 $5000\Delta t$ ，且同段互斥。

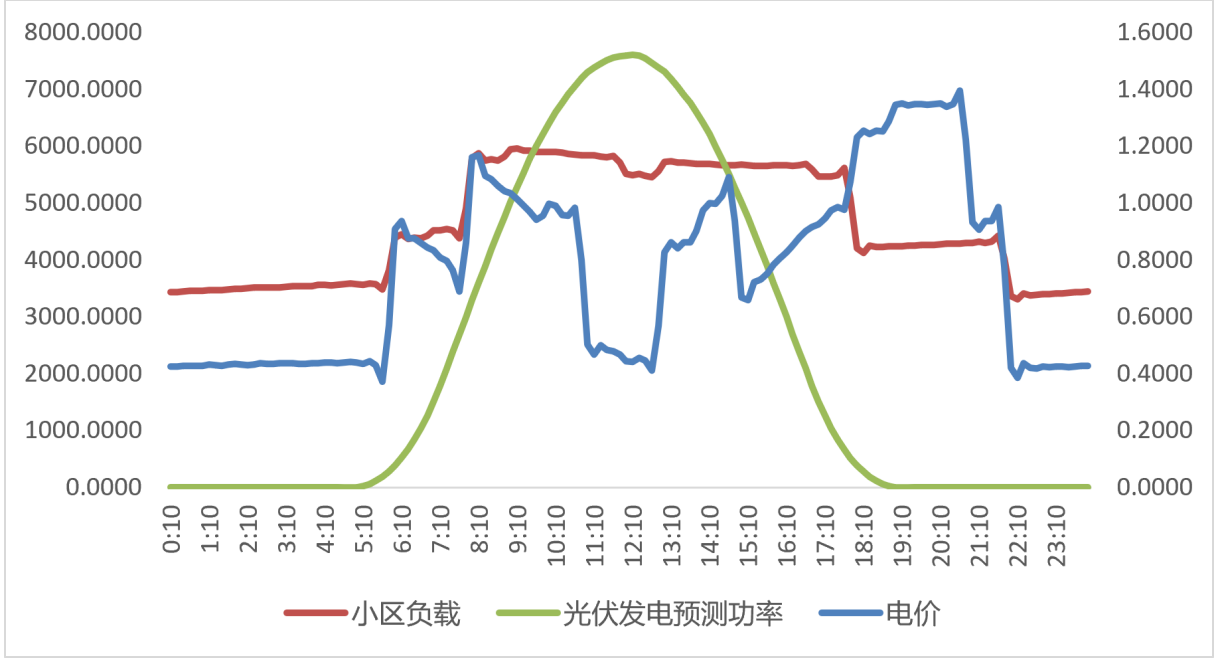


图 1 典型日负载、发电与电价

状态转移与购电决策。将储电量按步长 ΔE 离散为网格，保留上下界和精确初值。对可行转移 $E \rightarrow E'$ ，充放电动作及最小购电量为

$$\begin{aligned} c(E, E') &= \frac{(E' - E)^+}{\eta_c}, & d(E, E') &= \eta_d(E - E')^+, \\ g_t(E, E') &= [n_t + c(E, E') - d(E, E')]^+. \end{aligned} \quad (2)$$

其中， $(x)^+ = \max(x, 0)$ ；仅保留满足功率限额的转移，充放电互斥由转移方向保证。令 $a = \text{sgn}(E' - E)$ ，最近非空方向 $m \in \{-1, 0, 1\}$ 在空闲时不变，其余时更新为 a 。

目标函数与动态递推。以吞吐和方向反转刻画操作强度，阶段目标取

$$\ell_t(E, E', m) = p_t g_t(E, E') + \lambda_H(c + d) + \lambda_R \mathbb{I}(ma = -1). \quad (3)$$

其中， λ_H 、 λ_R 为操作偏好权重，不计入实际购电费用。相应 Bellman 递推为

$$\begin{aligned} J_t(E, m) &= \min_{E' \in \mathcal{A}(E)} \{ \ell_t(E, E', m) + J_{t+1}(E', m') \}, \\ J_{T+1}(E, m) &= \begin{cases} 0, & E = 6000, \\ +\infty, & E \neq 6000. \end{cases} \end{aligned}$$

其中， $\mathcal{A}(E)$ 为可行后继状态集合。典型日循环反转通过枚举起始非空方向并匹配终端方向处理，避免将切换转移到午夜边界。

先令操作权重为零，得到同网格纯费用基准 C^* ；再比较不同权重下的费用与操作指标，仅接受满足下式的方案：

$$C = \sum_t p_t g_t \leq (1 + \delta)C^* + \epsilon_C. \quad (4)$$

其中, δ 为费用容忍参数, ϵ_C 为数值容差; 具体费用范围另行确定。运行效果同时报告循环净功率总变差

$$\mathcal{V} = \sum_{t=1}^T |P_t^{\text{bat}} - P_{t-1}^{\text{bat}}|, \quad P_t^{\text{bat}} = (c_t - d_t)/\Delta t, \quad P_0^{\text{bat}} = P_T^{\text{bat}}. \quad (5)$$

吞吐与反转惩罚不等同于功率爬坡约束。若将总变差直接纳入优化, 需扩展上一槽功率状态; 本文不据操作次数推算电池寿命。

5.2 问题一模型的求解

采用逆向递推与前向回溯求得 144 槽购电和充放电轨迹。主网格取 50 kWh, 并以 25 kWh 检查离散敏感性; 细化网格可扩大动作选择, 但增加计算量, 所得最优值仅针对离散模型。操作权重增大通常强化少周转、少反转的偏好, 其费用代价及实际功率变化须单独核验。最终按题目指定时段汇总购电和储能结果。

【待更新】 以下表格和调度图为历史线性规划结果, 仅保留作对照, 尚未按本节动态规划重算; 历史目标 F_2 与 MIP 间隙不属于本节 DP 指标。旧表中购电汇总也存在不同层次结果混用, 更新时须由同一 DP 轨迹统一导出。

表 2 历史线性规划求解结果 (待 DP 重算)

层次	购电费/元	购电量/kWh	F_2	MIP 间隙
第一层	34080.951400	57572.396296	0.056210610	0
第二层	34115.032451	57560.643804	0.036950810	0

表 3 指定时段购电量及全天购电结果 (待更新)

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.000000	12:00–12:10	480.412450	14:00–14:10	0.000000
16:00–16:10	445.431650	18:00–18:10	621.850212	20:00–20:10	2.246095
全天购电量: 57560.616077 kWh			全天购电费: 34115.032451 元		

表 4 储能分时充放电量与起止储电量（待更新）

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	5059.644256	0.000000	4:00–8:00	0.000000	7357.436146
8:00–12:00	5071.280472	1702.996983	12:00–16:00	4995.867450	0.000000
16:00–20:00	0.000000	6729.235006	20:00–24:00	5059.644256	2378.124655
0:00 储电量: 6000.000000 kWh			24:00 储电量: 6000.000000 kWh		

原结果中，10:00–10:10 和 14:00–14:10 无需购电，说明该方案下光伏与储能已覆盖当段负载；具体时段安排待动态规划重算后核对。四小时内既有充电量又有放电量，是不同十分钟动作相加的结果，并不表示同一时刻同时充放电。

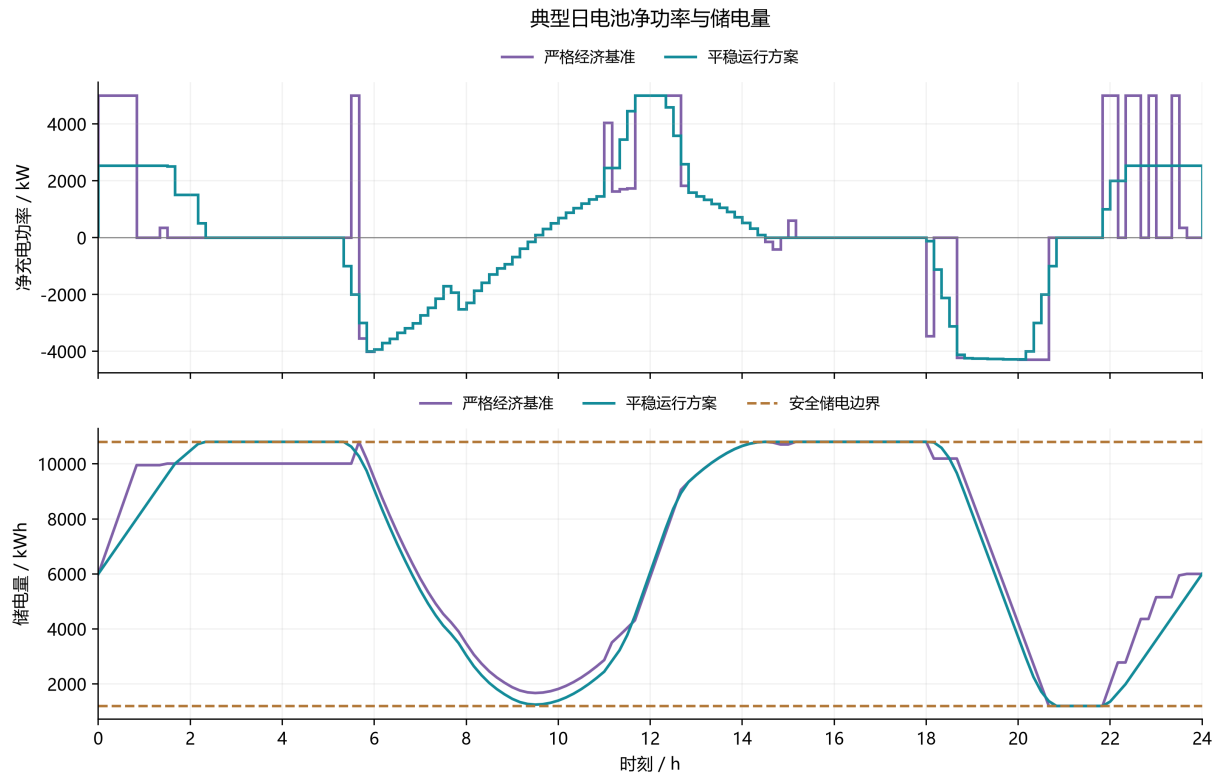


图 2 典型日电池净充电功率与储电轨迹（待更新）

图注：历史线性规划轨迹，仅作参考对照；新版动态规划曲线待统一重算。

表 5 储能调度方案的经济性与运行指标对比（待更新）

指标	严格经济基准	平稳运行方案
购电费/元	33801.495642	34115.032451
购电量/kWh	57526.243481	57560.616077
循环功率总变差/kW	102684.580769	33209.310058
最大相邻净功率变化/kW	8546.841200	1000.000000
运行段启动次数/次	12	4
短于 30 分钟的运行段数/段	6	0
转换损耗/kWh	1984.271048	2018.643643

原结果中，平稳方案相较严格经济基准，总变差降低 67.66%，启动次数减少 8 次，购电费增加 313.54 元，增幅约 0.93%。这些数值仅对应历史线性规划，不能作为新动态规划的计算结果。比较的重点是减少功率波动需要付出多少费用，而不是直接据此判断电池能够延长多少寿命；短动作减少也同时受到最短运行时间约束的影响。

六、问题二的模型建立与求解

问题二中，负载与光伏出力均随时间变化，而计划购电量必须在每日 0 时确定。供电不足将产生 5 倍电价的紧急购电费用，未使用的计划电量仍需支付费用，因此，预测偏差的方向及储能的跨时段调节能力共同影响购电成本。针对上述特点，建立基于条件误差决策树与离散储能动态规划的购电调度模型：首先预测当日负载和光伏功率，再利用历史预测误差估计条件净需求分布，进而优化日前购电与储能意图；实际运行时锁定购电计划，根据当槽供需与真实储电量调整充放电动作。

6.1 负载与光伏功率预测

6.1.1 输入特征与周期基线

沿用公共符号， d 表示日期， $t = 1, \dots, T$ 表示日内区间， $T = 144$ ， $\Delta t = 1/6 \text{ h}$ 。以 $a \in \{\text{L}, \text{V}\}$ 区分负载与光伏，记 $Y_{d,t}^{\text{L}} = L_{d,t}$ 、 $Y_{d,t}^{\text{V}} = V_{d,t}$ 。预测仅使用目标日 0 时以前的实际记录，不引入未来气象或全年季节信息。

居民用电具有日内与周内规律，光伏出力则具有连续日期间的日照周期特征。为保留这些先验信息，分别以负载上周同期值和光伏昨日同期值构造基础曲线：

$$B_{d,t}^{\text{L}} = L_{d-7,t}, \quad B_{d,t}^{\text{V}} = V_{d-1,t}. \quad (6)$$

模型在基础曲线上学习残差修正，不另叠加年度趋势模块。历史输入取最近 7 个完整日，将每 6 个十分钟功率值取均值，得到 168 点小时序列。小时聚合仅用于提取历史特征，预测输出仍保持十分钟分辨率。输入信息及其作用见表 ??。

表 6 负载与光伏预测的输入信息

信息类别	构造方式	作用
历史序列	过去 7 日的 168 点小时平均功率	提取近期水平及局部变化特征
同期信息	基础曲线、昨日值、上周值、过去 7 日平均功率	表征历史参照与日际偏移
日历信息	日内时刻与星期的正弦、余弦编码	保留周期相位，避免数值编码断点
固定电价	仅用于预测损失加权及后续调度	区分不同时段供电不足的经济后果

6.1.2 卷积残差网络与预测输出

负载与光伏采用结构相同、参数独立的一维卷积分支。历史序列经两层因果膨胀卷积、平均池化及全连接映射形成 8 维历史摘要，再与目标时刻的 8 维上下文拼接，输出标准化残差。两层卷积均取 8 个通道、核长 5，膨胀率分别为 1 和 2；池化长度与步长均为 6，预测头隐藏层宽度为 16，采用 ReLU 激活。

设第 k 月训练历史的均值与尺度为 μ_k^a 、 S_k^a ，其中 S_k^a 取历史标准差与 1 kW 的较大值。标准化历史序列记为 $\mathbf{X}_d^a$ ，上下文记为 $\mathbf{c}_{d,t}^a$ ，则网络映射与最终预测为

$$\begin{aligned}
\hat{r}_{d,t}^a &= f_{\Theta_k^a}(\mathbf{X}_d^a, \mathbf{c}_{d,t}^a), \\
\hat{L}_{d,t} &= \max\{0, B_{d,t}^L + S_k^L \hat{r}_{d,t}^L\}, \\
\hat{V}_{d,t} &= \mathbf{M}_{d,t} \max\{0, B_{d,t}^V + S_k^V \hat{r}_{d,t}^V\}.
\end{aligned} \tag{7}$$

式中： Θ_k^a 为网络参数； $\mathbf{c}_{d,t}^a$ 由表 ?? 中 4 项同期信息标准化后与 4 项日历编码组成； $\mathbf{M}_{d,t}$ 为光伏可发电时段掩码。该掩码取过去 28 个完整日正发电时段的并集，并向两侧各扩展 20 min，集合内取 1，其余取 0。两个分支合计 4930 个参数，输出层零初始化，使训练从基础曲线开始。每日 0 时输出 144×2 功率矩阵，供后续规划换算为区间电量。

6.1.3 非对称损失与时间顺序训练

负载低估或光伏高估均可能造成供电准备不足，故训练采用价格加权的非对称 Huber 损失。定义真实标准化残差 $r_{d,t}^a = (Y_{d,t}^a - B_{d,t}^a)/S_k^a$ 及误差 $u_{d,t}^a = r_{d,t}^a - \hat{r}_{d,t}^a$ ，方向权重为

$$\kappa^L(u) = 1 + 4\mathbb{I}(u > 0), \quad \kappa^V(u) = 1 + 4\mathbb{I}(u < 0). \tag{8}$$

式中， $\mathbb{I}(\cdot)$ 为指示函数。两类供电不足方向的权重为 5，反方向为 1。令 $\mathcal{T}_k$ 为训练日期集合， D_k 为训练段的排他截止日，采用 90 日半衰期的历史权重：

$$\omega_{k,d} = \frac{2^{-(D_k-1-d)/90}}{|\mathcal{T}_k|^{-1} \sum_{j \in \mathcal{T}_k} 2^{-(D_k-1-j)/90}}, \quad \bar{p} = \frac{1}{T} \sum_{t=1}^T p_t. \quad (9)$$

两个分支的联合训练目标写为

$$\mathcal{L}(\Theta) = \frac{1}{2T|\mathcal{T}_k|} \sum_{d \in \mathcal{T}_k} \omega_{k,d} \sum_{t=1}^T \sum_{a \in \{L,V\}} \frac{p_t}{\bar{p}} \kappa^a(u_{d,t}^a) \mathcal{H}(u_{d,t}^a) + 10^{-4} \sum_{W \in \mathcal{W}} \|W\|_F^2. \quad (10)$$

式中： $\mathcal{H}(u)$ 在 $|u| \leq 1$ 时为 $u^2/2$ ，否则为 $|u| - 1/2$ ； $\mathcal{W}$ 为两分支卷积核集合。该目标是供电风险的训练代理，尚未计入储能状态与跨时段调度影响，因而不等同于真实购电费用。

网络按月从头训练，月初以前最近 7 个完整日用于早停，其余历史日用于训练；标准化统计量仅由训练段计算，训练样本的目标窗口不得跨入验证段。采用 Adam 算法，学习率 0.001、批大小 32、最多 60 轮，验证损失连续 6 轮不改善时停止，并恢复最佳参数。当月参数冻结，每日只更新历史输入和光伏掩码。正式比较固定种子 42，其他种子用于稳定性检验，不按评价期成绩选取最优种子。

6.2 基于决策树的条件净需求分布

6.2.1 历史误差样本与条件划分

点预测不能充分反映供需偏差对紧急购电的影响。为使日前计划考虑误差幅度，先将功率转换为区间净需求电量：

$$n_{d,t} = (L_{d,t} - V_{d,t})\Delta t, \quad \hat{n}_{d,t} = (\hat{L}_{d,t} - \hat{V}_{d,t})\Delta t, \quad \epsilon_{d,t} = n_{d,t} - \hat{n}_{d,t}. \quad (11)$$

净需求允许为负； $\epsilon_{d,t} > 0$ 表示预测低估净需求。每天 0 时，从最近至多 28 个已完整揭晓的历史日收集 $(\mathbf{z}_{j,t}, \epsilon_{j,t})$ ，其中

$$\mathbf{z}_{d,t} = \left(\sin \frac{2\pi(t-1)}{T}, \cos \frac{2\pi(t-1)}{T}, \hat{n}_{d,t}, \hat{V}_{d,t}\Delta t, p_t \right)^\top. \quad (12)$$

特征分别表征日内相位、净需求水平、光伏规模及用电时段。以误差为回归目标，按平方误差下降准则递归划分样本，树深上限为 5，每个叶节点至少包含 48 个槽位样本，随机状态固定为 42。树的作用是形成条件相近的误差样本集合，后续使用叶内分位数，而非仅以叶内均值替代全部不确定性。

当有效预测发布日不足两个时，采用历史周期预测误差回退：负载取昨日与上周同槽均值，光伏取昨日同槽值；不足一周时负载采用昨日值。此时明确区分周期基线误差与网络预测误差。只有某日实际值完整揭晓后，才将该日残差加入后续样本，避免使用未来观测。

6.2.2 九点边际支持

设目标槽所属叶节点的历史误差经验分位函数为 $Q_{\mathcal{A}_{d,t}}$ ，构造

$$q_j = \frac{j - \frac{1}{2}}{9}, \quad x_{d,t,j} = \hat{n}_{d,t} + Q_{\mathcal{A}_{d,t}}(q_j), \quad \pi_j = \frac{1}{9}, \quad j = 1, \dots, 9. \quad (13)$$

式中，叶内分位数采用线性插值， $x_{d,t,j}$ 为净需求支持点，单位为 kWh。由此以有限支持近似每槽的条件净需求分布，为规划阶段提供误差幅度信息。九列支持分别对应各槽边际分位点，不表示九条相关的整日天气路径；连续缺口的完整时间相关性未在该近似中显式保留。

6.3 日前购电与储能动态规划

6.3.1 储能状态与可行转移

规划阶段省略日下标 d 。沿用 E_t 表示区间 t 开始时的内部储电量， E_{T+1} 表示日末值；以 c_t^0, d_t^0 表示计划交流侧充、放电量，上标 0 用于与实际动作区分。令 c_t, d_t 为对应实际电量，两类动作均受公共设备参数约束。对于网格转移 $E \rightarrow E'$ ，定义

$$c^0(E, E') = \frac{(E' - E)_+}{\eta_c}, \quad d^0(E, E') = \eta_d(E - E')_+. \quad (14)$$

式中， $(x)_+ = \max(x, 0)$ ， $\eta_c = \eta_d = \sqrt{0.9}$ 。单一转移仅可能充电、放电或空闲，因而自然满足互斥。可行后继状态集合为

$$\mathcal{S}(E) = \{E' \in \mathcal{G} : 0 \leq c^0(E, E'), d^0(E, E') \leq P_{\max} \Delta t\}. \quad (15)$$

其中， $\mathcal{G} \subset [1200, 10800]$ 为储电量网格，基础间距取 50 kWh，并插入精确日初储电量及上下界，不对实际初态作四舍五入；另以 25 kWh 网格检验离散敏感性。

为统计充放方向反转，在状态中增加最近一次非空动作方向 $m \in \{-1, 0, 1\}$ ，分别表示放电、尚无非空动作和充电。若 $a = \text{sign}(E' - E)$ ，方向转移为

$$m' = \begin{cases} m, & a = 0, \\ a, & a \neq 0. \end{cases} \quad (16)$$

空闲时保留原方向，使停机后恢复同方向不被计为反转。

6.3.2 非对称购电费用与分位数决策

给定计划动作 c^0, d^0 ，以有限支持近似紧急购电的期望代价，得到阶段代理费用

$$F_t(g; c^0, d^0) = p_t \left[g + 5 \sum_{j=1}^9 \pi_j (x_{t,j} + c^0 - d^0 - g)_+ \right], \quad g \geq 0. \quad (17)$$

令 $Z_t = X_t + c^0 - d^0$, G_{Z_t} 为其离散分布函数。当 $g > 0$ 时, 凸函数最优性条件为 $G_{Z_t}(g^-) \leq 4/5 \leq G_{Z_t}(g)$, 从而可取

$$g_t^*(c^0, d^0) = \max\{0, Q_{0.8}(X_t) + c^0 - d^0\}. \quad (18)$$

该分位水平由计划电价与 5 倍紧急电价的边际权衡导出。离散分位数定义为 $Q_\alpha(X) = \inf\{x : G_X(x) \geq \alpha\}$; 九个等权支持下取第八小值, 对应式 (??) 中叶内 5/6 分位点, 因此压缩后并非原始叶分布精确的 0.8 分位。

式 (??) 仅对固定计划动作的代理目标成立。实际运行允许电池随观测改变动作, 故该式不构成真实反馈费用的精确最优解; 其作用是消去规划中的连续购电变量, 降低动态规划的计算规模。

6.3.3 操作偏好与 Bellman 递推

为兼顾电池操作强度, 引入交流侧吞吐与方向反转的阶段代价

$$\ell_{\text{op}}(E, E', m) = \lambda_H[c^0(E, E') + d^0(E, E')] + \lambda_R \mathbb{I}\{m \text{ sign}(E' - E) = -1\}. \quad (19)$$

参考设置为 $\lambda_H = 0.002$ 元/kWh、 $\lambda_R = 0.05$ 元/次。这两项是规划偏好, 不计入题目购电费用, 也不解释为实测电池折旧单价。

记 $J_t(E, m)$ 为从区间 t 至日末的最小代理代价, 将式 (??) 代回 F_t , 得到 $F_t^*(E, E')$, 递推为

$$J_t(E, m) = \min_{E' \in \mathcal{S}(E)} \{F_t^*(E, E') + \ell_{\text{op}}(E, E', m) + J_{t+1}(E', m')\}. \quad (20)$$

普通日以剩余储电量的近似价值连接次日, 终端条件取

$$J_{T+1}(E, m) = -\frac{\min_t p_t}{\eta_c}(E - 1200). \quad (21)$$

评价期最后一天取 $J_{T+1} = 0$ 。终端价值仅用于规划, 不从真实账单中抵扣; 第二问不施加日初日末储电量相同的约束。由 $t = T$ 向前递推并保存最优后继状态, 再从真实日初状态正向回溯, 得到全天购电量与储能意图。

上述递推对给定网格及阶段代理求优, 不提供原连续随机运行问题的全局最优证书。状态 (E, m) 只能精确统计吞吐与方向反转; 若将相邻功率变化纳入优化, 还需增加上一槽功率状态, 不能仅凭方向变量声称已直接优化功率总变差。

6.4 基于实际供需的储能执行与费用核算

6.4.1 允许方向改变的实时执行

日前购电量 g_t 确定后全天保持不变。第 t 槽实际负载和光伏揭晓后, 计算购电与发电相对负载的净剩余

$$B_t = g_t + (V_t - L_t)\Delta t = g_t - n_t. \quad (22)$$

若 $B_t \geq 0$ ，将可用剩余电量用于充电；若 $B_t < 0$ ，优先使用电池补足缺口。实际动作分别为

$$c_t = \min \left\{ B_t, P_{\max} \Delta t, \frac{E_{\max} - E_t}{\eta_c} \right\}, \quad d_t = e_t = 0, \quad (23)$$

$$w_t = B_t - c_t, \quad B_t \geq 0;$$

$$d_t = \min \{ -B_t, P_{\max} \Delta t, (E_t - E_{\min}) \eta_d \}, \quad c_t = w_t = 0, \quad (24)$$

$$e_t = -B_t - d_t, \quad B_t < 0.$$

实际储电量递推与能量平衡为

$$E_{t+1} = E_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad g_t + e_t + d_t = n_t + c_t + w_t. \quad (25)$$

因此，实际充放电方向不受日前意图方向限制，但始终服从储电量、功率和互斥条件。紧急购电仅补充供电缺口，不主动用于充电； w_t 为总未利用电量，可能包含已付费剩余购电。本执行基线不设置 2 kWh 死区，以免主动放弃小额充电或缺口补偿。

由于实际动作可能偏离规划，次日必须以本执行器的真实日末储电量和最近非空方向重新规划，不能用计划末状态替代。此时操作偏好对实际反转与吞吐的影响须通过回放检验，不能由规划目标直接推定。

6.4.2 真实费用与操作指标

在评价日期集合 $\mathcal{D}$ 上，真实购电总费为

$$C_{\text{actual}} = \sum_{d \in \mathcal{D}} \sum_{t=1}^T (p_t g_{d,t} + 5p_t e_{d,t}). \quad (26)$$

计划电量无论是否使用均全额计费，不包含日内调整费、辅助罚项或终端残值。为避免仅用费用评价电池运行，另计算交流侧吞吐 H 、净功率总变差 TV 及电芯侧等效满循环数 N_{EFC} ：

$$\begin{aligned} H &= \sum_{d,t} (c_{d,t} + d_{d,t}), & P_{d,t}^{\text{bat}} &= \frac{c_{d,t} - d_{d,t}}{\Delta t}, \\ TV &= \sum_{i=2}^N |P_i^{\text{bat}} - P_{i-1}^{\text{bat}}|, & N_{\text{EFC}} &= \frac{\eta_c \sum_{d,t} c_{d,t} + \sum_{d,t} d_{d,t} / \eta_d}{2E_{\text{cap}}}. \end{aligned} \quad (27)$$

式中： $d_{d,t}$ 为日期 d 、区间 t 的实际放电量； $E_{\text{cap}} = 12000 \text{ kWh}$ ； i 将评价期 $N = 48096$ 个槽位按时间连续编号。方向反转次数另在剔除空闲槽后统计相邻非空动作符号改变，包含评价期内部跨日连接，不含一月预热边界。上述指标分别反映能量周转、功率变化和方向切换，不能直接换算为电池寿命增益。

若需控制操作偏好的经济代价，可在相同信息、执行规则和评价区间下，以纯费用对照为参照设置

$$C_{\text{actual}} \leq (1 + \varepsilon_C) C_{\text{ref}}. \quad (28)$$

其中 ε_C 为方案筛选条件，而非数值求解容差；其取值及是否允许正增幅应在新实验前确定。操作权重的软惩罚不能保证该费用界，须用真实执行费用另行核验，历史验证通过也不提供未来费用保证。本文不将尚未确定的费用比例写成既定参数。

6.5 求解流程与已有算例分析

6.5.1 逐日求解流程

采用冻结预测、逐日更新误差分布和逐槽执行的时间顺序回放。1月1日从6000kWh预热；为与已有实验保持相同初态，2月1日储电量约为1421.799111kWh，计算时保留预热档案的原始精度，此后各策略连续传递自己的真实状态。该初态来自已有预热，不视为本节新模型的独立运行结果。具体步骤如下。

- 1) 在0时读取实际日初状态及已冻结网络，生成当日144点负载和光伏预测。
- 2) 仅用此前完整历史日更新条件误差树，构造每槽九点净需求支持。
- 3) 建立SOC网格及可行转移，依据式(??)逆向求解，正向回溯并锁定全天购电量。
- 4) 逐槽读取实际供需，按式(??)、式(??)确定动作，记录紧急购电、剩余电量和实际SOC。
- 5) 当日结束后核算费用与运行指标，将真实末状态及已完整揭晓的误差传递至下一日。

对全部评价槽检查能量平衡、SOC递推、边界、充放电互斥和购电锁定；动态规划以小规模穷举与25/50kWh网格对照检验递推及离散影响。最终按题目要求汇总四个指定日的购电、充放电与紧急购电记录，并保存完整result2.xlsx。

6.5.2 已有方案对照与适用范围

为说明规划与执行方式的影响，表??列出统一建模方案所汇总的已有334日对照。三组使用相同的无年度趋势修正预测输入与物理结算口径；其中，意图裁剪组限制实际动作不超过日前意图，实时响应组则采用式(??)、式(??)，并按各自实际末状态逐日重规划。

表7 相同预测输入下的已有购电与运行指标对照

方案	总费用/万元	非空反转/次	总变差/万kW
基础调度	1361.1588	7715	3343.0433
树DP+意图裁剪	1406.6257	2729	3045.1039
树DP+实时响应及重规划	1353.4197	6755	3266.6146

已有结果表明，实时响应及重规划组相对基础调度节省77391.11元，降幅约0.57%，

非空反转减少 12.44%，总变差下降约 2.29%。意图裁剪组虽使反转减少 64.63%，却增加 3.34% 的购电费用。两类结果说明，对充放电意图施加较强限制可能改善部分操作指标，但未必取得更低的真实购电成本，故本节采用允许实际方向改变的执行基线。

表中数值为已有实验记录，并非本次费用容忍条件和操作权重重新筛选后的结果；实时响应组目前属于单种子对照。其费用与运行改善尚需在固定参数后补充稳定性验证，不能把意图裁剪组的反转降幅移用于新执行方式。模型的主要局限在于逐槽边际支持未充分刻画连续误差相关性，且计划动作与实际反馈存在差异；因此，最终方案应依据真实费用和实际运行指标共同判断，而不以代理目标或单项平稳性指标替代综合评价。

七、 问题三的模型的建立和求解

7.1 具体分析

7.2 模型准备

7.3 模型建立

7.4 模型求解

八、 问题四的模型的建立和求解

8.1 具体分析

8.2 模型准备

8.3 模型建立

8.4 模型求解

九、模型的分析与检验

9.1 模型检验与敏感性分析

为区分储能收益和运行偏好的代价，设置三个对照：无储能直接购电；不增加爬坡及持续时间要求的严格经济基准；采用 $R_{\text{ramp}} = 1000$ 、 $U = 3$ 、 $D = 2$ 、 $\delta = 0.001$ 的平稳方案。严格经济基准同样采用两层求解，但第二层只保留数值费用容差，不另增加费用预算。无储能方案为

$$g_t^{(0)} = (L_t - V_t)^+ \Delta t, \quad C^{(0)} = \sum_t p_t g_t^{(0)} = 48052.046591 \text{ 元}. \quad (29)$$

其中， $g_t^{(0)}$ 为无储能条件下区间 t 直接从外网购买的电量，单位为 kWh； $C^{(0)}$ 为该无储能方案的全天购电费用，单位为元；上标 (0) 表示无储能对照方案，取正部表示光伏超过负载时购电量为零。各项指标见表 ??，此处不再重复列出。

【待更新】 本节比较指标、敏感性试验和噪声试验暂保留原三层程序的结果及分析。待程序改为两层后，重新计算表格、曲线与变化幅度，并核对以下结论。

原计算结果中，相较严格经济基准，平稳方案的总变差降低 67.66%，启动次数减少 8 次。购电费增加 313.54 元，增幅约 0.93%，其经济代价可分解为

$$\underbrace{C_{\text{stable}} - C_{\text{econ}}^*}_{\text{总经济代价}} = \underbrace{C^* - C_{\text{econ}}^*}_{\text{新增运行限制}} + \underbrace{C_{\text{stable}} - C^*}_{\text{第二层费用让步}}. \quad (30)$$

其中， C_{stable} 为第二层求解后平稳运行方案的实际全天购电费用， C_{econ}^* 为不增加爬坡及最短持续时间限制时严格经济基准的第一层最优费用，单位均为元； C^* 仍为加入运行限制后的第一层最优费用。原结果中两项分别约为 279.455857 元和 34.081051 元（待按两层结果核对），其中 C_{econ}^* 为严格经济基准第一层最优值。由此可见，0.1% 指新运行条件下的额外费用预算，而非相对原方案的全部费用增幅。

9.1.1 费用让步比例的选取

固定其他设置，在 $\delta = 0, 0.0005, 0.001, 0.002, 0.005$ 上分别执行两层优化，待更新的原结果见表 ??。将有效运行且净功率绝对值小于 50 kW 的区间称为低功率段，仅用于诊断短小动作，不另作硬约束。

表 8 电费让步比例的单因素重求解结果（待更新）

$\delta/\%$	购电费/元	购电量/kWh	总变差/kW	启动数	低功率段数
0	34080.951500	57572.396243	50937.422410	6	4
0.05	34097.991975	57570.793390	34345.144627	6	1
0.10	34115.032451	57560.616077	33209.310058	4	0
0.20	34149.113402	57552.981594	31090.138980	4	0
0.50	34251.356257	57516.782839	28513.498282	4	0

原参数试验中，由 0 提高至 0.05% 已大幅降低总变差，但启动仍为 6 次；提高至 0.10% 后，启动降至 4 次且无低功率段。继续增加预算仍可降低总变差，却未继续减少启动。因此暂沿用 0.10% 作为本组候选中的折中值，待两层试验完成后核对，而不将其解释为连续参数空间中的唯一最优点。

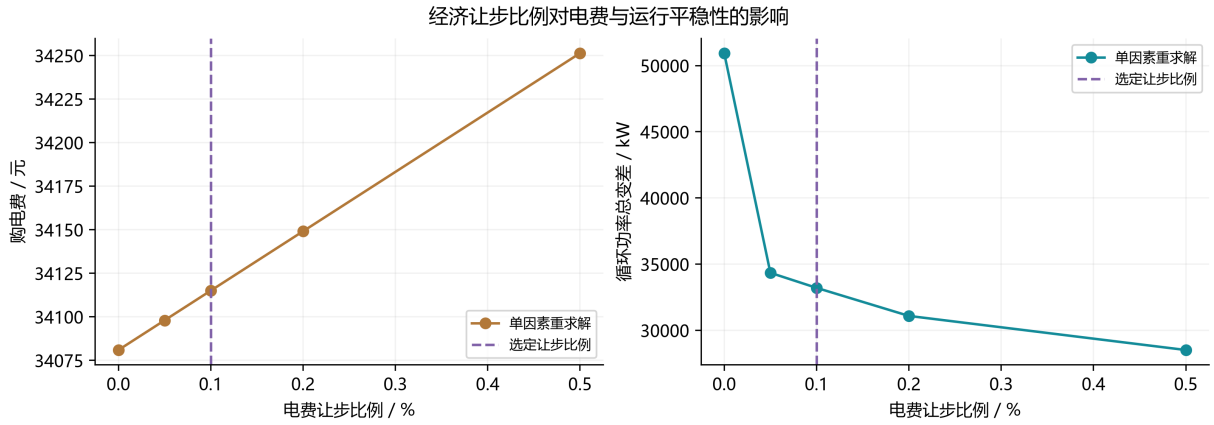


图 3 费用让步比例与运行平稳性的关系（待更新）

图注：小幅费用让步可明显降低功率总变差；预算继续增加时启动次数不再下降，需结合额外支出选择运行方案。

9.1.2 核心参数的局部扰动检验

对 $\delta, R_{\text{ramp}}, \eta_{\text{rt}}$ 分别作 $\pm 10\%$ 相对扰动，其余参数不变，每种情景均按两层模型重新求解。定义指标 J 的相对变化及局部灵敏度为

$$D_J(\theta') = \frac{J(\theta') - J(\theta)}{J(\theta)} \times 100\%, \quad S_J(\theta') = \frac{[J(\theta') - J(\theta)]/J(\theta)}{(\theta' - \theta)/\theta}. \quad (31)$$

其中， θ 为被扰动参数的基准值， θ' 为扰动后的取值； J 为所考察的输出指标，如购电费或功率总变差， $J(\theta)$ 与 $J(\theta')$ 分别为扰动前、后重新求解得到的指标值。 $D_J(\theta')$ 为该指标的百分比变化， $S_J(\theta')$ 为指标相对变化与参数相对变化之比，是无量纲的有限扰动灵敏度。整数模式可能跳变，故 S_J 用于有限扰动比较，不等同于处处存在的导数。这

里 δ 的 $\pm 10\%$ 是 0.09% 与 0.11%，效率的对应值是 81% 与 99%，后者为机制压力情景，不表示实测效率误差范围。

表 9 核心参数相对变化 10% 时的局部敏感性（待更新）

参数情景	购电费/元	费用变化/%	总变差/kW	启动数
$\delta = 0.09\%$	34111.6244	-0.0100	34436.1963	4
$\delta = 0.11\%$	34118.4405	0.0100	32900.5165	4
$R_{\text{ramp}} = 900 \text{ kW}$	34153.5161	0.1128	33045.9374	4
$R_{\text{ramp}} = 1100 \text{ kW}$	34085.2443	-0.0873	33952.8546	4
$\eta_{\text{rt}} = 81\%$	35470.6053	3.9735	32667.2116	4
$\eta_{\text{rt}} = 99\%$	32897.4785	-3.5690	33591.6607	6

原计算结果中，费用让步比例的局部变化仅使电费变动约 $\pm 0.0100\%$ ，启动数均为 4；但总变差与低功率段仍会变化，说明局部轨迹并非完全不敏感。爬坡限值由 1000 降低至 900 kW，费用增加约 0.1128%；放宽至 1100 kW，费用下降约 0.0873%，体现了连续运行与经济支出的权衡。

相比之下，往返效率降至 81% 时费用增加约 3.9735%，升至 99% 时费用下降约 3.5690%，且后者启动次数增至 6。效率影响单位放电的购入成本，还会改变有利套利时段和模式选择。因此只能认为本轮方案对费用预算及爬坡参数的局部扰动较稳定，不能据此声称对全部参数均不敏感。原试验中各情景的独立约束违反项均为 0，两层模型的结果仍需重新检验。

9.1.3 输入噪声与固定计划压力测试

在缺乏实测误差分布的条件下，设置小幅、可重复的数值扰动，检验模型对输入变化的反应：

$$\begin{aligned}\tilde{L}_t &= L_t(1 + \zeta_t^L), & \tilde{V}_t &= V_t(1 + \zeta_t^V), \\ \zeta_t^j &= \text{clip}(Z_t^j, -0.03, 0.03), & Z_t^j &\sim N(0, 0.01^2).\end{aligned}\tag{32}$$

其中， $\tilde{L}_t$ 与 $\tilde{V}_t$ 分别为扰动后的负载功率与光伏功率，单位为 kW； ζ_t^L 与 ζ_t^V 为相应的无量纲相对扰动。此式 $j \in \{L, V\}$ 表示扰动对象， Z_t^j 为均值 0、标准差 0.01 的正态随机扰动， $N(0, 0.01^2)$ 中的第二个参数为方差；clip 将扰动截断在 $[-0.03, 0.03]$ ，即 $\pm 3\%$ 以内。使用种子 2026、2027、2028，保持电价不变、夜间零光伏不变，各情景按两层模型独立重求解。同时冻结原购电和充放电指令，计算其不能覆盖的正向净负荷扰动：

$$H_{\text{fixed}} = \sum_t \left[(\tilde{L}_t - \tilde{V}_t - L_t + V_t) \Delta t - w_t \right]^+.\tag{33}$$

其中， H_{fixed} 为冻结原购电及充放电指令后，全天各区间供电缺口的累计电量，单位为 kWh；方括号内为净负荷新增电量减去原方案剩余电量 w_t ，外侧取正部表示只累计未能覆盖的缺口，不用其他区间的富余电量直接抵消。 H_{fixed} 是不允许重新调度或补购时的计划缺口，不是采用实时补救后必然发生的停电量。两项检验分别回答“重新优化是否稳定”与“固定计划能否直接应对扰动”。

表 10 1% 尺度输入扰动下的重求解与固定计划检验（待更新）

随机种子	重求解费用/元	费用变化/%	固定计划缺口/kWh	重求解违反项
2026	34143.6909	0.0840	609.1943	0
2027	34304.0402	0.5540	723.4288	0
2028	34058.5619	-0.1655	470.7939	0

原三次重求解的费用变化处于 -0.1655% 至 0.5540% ，全部通过约束复核，说明本组小幅扰动下重新规划仍能得到接近原费用的可行解。然而，冻结原计划后累计出现 $470.7939\text{--}723.4288\text{ kWh}$ 的区间缺口，表明确定性计划不具备自动抵御供需误差的保供保证。该结果也说明，后续引入实时补购与滚动调整具有必要性。三次独立高斯扰动仅检验局部数值稳定性，不代表真实误差具有独立正态分布，也不提供极端天气或相关预测偏差下的概率保证。

9.2 灵敏度分析

9.3 误差分析

十、模型的评价、改进与推广

10.1 模型的优点

- 优点 1
- 优点 2
- 优点 3

10.2 模型的缺点

- 缺点 1
- 缺点 2

10.3 模型的改进

10.4 模型的推广

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型思路讨论、程序编写与调试、结果分析辅助、论文表述与排版调整，详细使用情况见支撑材料。

参考文献

- [1] HUANG W, YE Y, CHEN H, et al. Onboard early detection and mitigation of lithium plating in fast-charging batteries[J]. Nature Communications, 2022, 13: 7091. DOI: <https://doi.org/10.1038/s41467-022-33486-4>.
- [2] XU B, ZHAO J, ZHENG T, et al. Factoring the Cycle Aging Cost of Batteries Participating in Electricity Markets[EB/OL]. arXiv:1707.04567, 2017. <https://arxiv.org/abs/1707.04567>.
- [3] BABIĆ L, LAURICELLA M, CEUSTERS G, et al. Data-driven non-parametric chance-constrained model predictive control for microgrids energy management using small data batches[J]. Frontiers in Control Engineering, 2023, 4: 1237759. DOI: <https://doi.org/10.3389/fcteg.2023.1237759>.
- [4] GESLIN A, XU L, GANAPATHI D, et al. Dynamic cycling enhances battery lifetime[J]. Nature Energy, 2025, 10: 172–180. DOI: <https://doi.org/10.1038/s41560-024-01675-8>.
- [5] SciPy contributors. scipy.optimize.milp: Mixed-integer linear programming[EB/OL]. SciPy 1.15.3 documentation. <https://docs.scipy.org/doc/scipy-1.15.3/reference/generated/scipy.optimize.milp.html>.

附录 A 文件列表

文件名	功能描述
q1_reproduce.py	问题一原求解、检验与绘图程序（待同步为两层）
数据预处理.py	预处理程序代码
q2.py	问题二程序代码
q3.c	问题三程序代码
q4.cpp	问题四程序代码
AI 工具使用详情.pdf	AI 工具、使用过程及人工修改与核验情况说明（提交前补齐）

附录 B 问题一完整求解与检验程序

【待更新】 以下暂保留原三层程序，尚未与正文的两层模型同步。后续需删除最小购电量目标，以第二层轨迹作为最终输出，并同步约束复核、汇总和绘图，再统一更新正文数值。

原程序包含原始输入、三层求解、独立约束检验、参数敏感性、噪声压力测试及绘图。数据取自附件一的 144 段记录，程序不依赖个人绝对路径。将以下代码保存为 `q1_reproduce.py`，安装依赖后直接执行：

```
python -m pip install numpy scipy matplotlib
python q1_reproduce.py
```

若希望读取外部 Excel，另安装 `openpyxl`，并使用 `--input` 传入文件名。外部文件损坏或缺失时默认停止，只有显式加入 `--fallback` 才回退到内嵌数据，避免悄悄替换输入。

`--quick` 只运行主方案与严格经济基准；默认运行 12 组参数方案和 3 组噪声方案。

输出目录默认为 `q1_output`，其中 `revised.csv` 包含 144 段完整购电、充放电及储电量；各情景的 JSON 文件保存三层求解状态与完整轨迹，`summary.json` 包含指定时段、四小时汇总及检验结果。CSV 可直接用表格软件打开，输入附件和已有结果工作簿均不修改。两幅 PNG 图由同一批计算结果绘制，当前正文暂存这些旧输出，待两层程序完成后统一替换。

```
1 """问题一完整复现: Python >=3.10; pip install numpy scipy matplotlib.
2 直接运行 python q1_reproduce.py; 输入内嵌为附件1原始144行。
3 可选 --input 附件1.csv / 附件1.xlsx; 损坏文件默认报错, --fallback才使用内嵌数据。
4 全部输出放入 --out 指定的新目录; 不覆盖原始附件或队友工作簿。
5 """
6 import argparse # 解析命令行参数。
7 import csv # 读写不依赖Excel软件的表格。
8 import io # 将内嵌文本作为文件读取。
9 import json # 保存完整数值和求解状态。
10 import os # 将绘图缓存限制在本次输出目录。
11 import sys # 向终端返回失败状态。
12 import time # 记录真实求解耗时。
13 from pathlib import Path # 使用跨平台路径, 不写入个人绝对路径。
14 try:
15     import numpy as np # 数值计算。
16     import scipy # 记录实际使用的库版本。
17     from scipy.optimize import milp, Bounds, LinearConstraint # 混合整数规划。
18     from scipy.sparse import coo_matrix # 稀疏约束矩阵。
19 except ImportError as exc:
20     raise SystemExit("请先安装: pip install numpy scipy matplotlib") from exc
21
22 # 以下是附件1逐行数据, 顺序为区间右端、电价、负载、光伏功率。
23 EMBEDDED_CSV = """时间,电价,小区负载,光伏发电预测功率
24 00:10:00,0.4248,3439.8466,0
25 00:20:00,0.4245,3437.9792,0
26 00:30:00,0.4269,3444.3031,0
27 00:40:00,0.4272,3454.4437,0
28 00:50:00,0.4271,3458.8273,0
29 01:00:00,0.4285,3459.8401,0
30 01:10:00,0.4311,3467.1582,0
31 01:20:00,0.4302,3467.9632,0
32 01:30:00,0.4279,3466.9683,0
33 01:40:00,0.4312,3482.4927,0
34 01:50:00,0.4333,3495.0841,0
35 02:00:00,0.4311,3498.4443,0
36 02:10:00,0.4293,3505.2295,0
37 02:20:00,0.4324,3512.6387,0
38 02:30:00,0.4367,3516.8685,0
39 02:40:00,0.4348,3517.9246,0
40 02:50:00,0.4335,3518.5518,0
```

41	03:00:00,0.4357,3521.0976,0
42	03:10:00,0.4374,3526.9261,0
43	03:20:00,0.4371,3537.2854,0
44	03:30:00,0.4356,3544.8808,0
45	03:40:00,0.4353,3541.2787,0
46	03:50:00,0.436,3546.3976,0
47	04:00:00,0.4378,3560.207,0
48	04:10:00,0.4401,3561.9581,0
49	04:20:00,0.4389,3556.7183,0
50	04:30:00,0.4364,3560.0272,0
51	04:40:00,0.4393,3580.654,0.5994
52	04:50:00,0.4422,3586.916,0.2345
53	05:00:00,0.4395,3571.2635,0.2885
54	05:10:00,0.4333,3562.3171,18.6781
55	05:20:00,0.4439,3591.3809,54.4698
56	05:30:00,0.4301,3579.7045,108.9718
57	05:40:00,0.3713,3486.119,184.1603
58	05:50:00,0.5708,3823.8994,277.0582
59	06:00:00,0.9099,4398.0727,388.5494
60	06:10:00,0.9368,4454.0123,521.2803
61	06:20:00,0.8758,4371.7919,672.7677
62	06:30:00,0.8759,4392.953,844.2261
63	06:40:00,0.8601,4384.5956,1039.6744
64	06:50:00,0.8442,4434.7757,1259.3375
65	07:00:00,0.8334,4523.4561,1508.6318
66	07:10:00,0.8092,4524.0721,1791.4107
67	07:20:00,0.7972,4545.8867,2085.4865
68	07:30:00,0.7644,4523.2418,2381.2611
69	07:40:00,0.6899,4383.3566,2683.1752
70	07:50:00,0.86,4910.4599,2986.3552
71	08:00:00,1.1617,5799.7327,3289.204
72	08:10:00,1.1683,5884.826,3593.3613
73	08:20:00,1.0956,5748.0713,3890.7332
74	08:30:00,1.0849,5769.0581,4181.3417
75	08:40:00,1.061,5754.2105,4471.7657
76	08:50:00,1.043,5824.1807,4752.8272
77	09:00:00,1.036,5954.2601,5021.9074
78	09:10:00,1.0147,5958.9696,5285.4126
79	09:20:00,0.9922,5929.5554,5543.1742
80	09:30:00,0.97,5923.7828,5788.4093
81	09:40:00,0.9422,5907.586,6012.1304
82	09:50:00,0.9569,5899.1262,6216.1528
83	10:00:00,0.9977,5902.2196,6409.5941
84	10:10,0.9919,5897.4793,6599.3808
85	10:20,0.9588,5886.7568,6766.7204
86	10:30,0.9549,5873.4708,6914.5923
87	10:40,0.9839,5858.0308,7064.2053
88	10:50,0.799,5844.9948,7197.4315
89	11:00,0.5025,5839.0587,7303.1142
90	11:10,0.4662,5838.7103,7387.6848
91	11:20,0.5007,5823.4378,7459.227
92	11:30,0.4827,5814.7932,7520.7191
93	11:40,0.4785,5832.3951,7568.2039
94	11:50,0.4662,5712.901,7592.1539
95	12:00,0.4432,5519.4607,7601.0433
96	12:10,0.4411,5494.7907,7612.316
97	12:20,0.4551,5514.4524,7602.6524
98	12:30,0.4463,5485.829,7554.1872
99	12:40,0.4113,5462.3628,7472.7384
100	12:50,0.5682,5566.1066,7393.4172
101	13:00,0.8279,5725.7548,7312.7979
102	13:10,0.8636,5739.4957,7194.541
103	13:20,0.8416,5714.6836,7053.1371
104	13:30,0.8618,5717.8505,6910.4064
105	13:40,0.8623,5708.2486,6772.4075
106	13:50,0.9026,5696.9182,6607.36
107	14:00,0.9754,5692.2983,6412.5085
108	14:10,1.0002,5687.1291,6214.0158
109	14:20,0.9978,5678.962,6000.7112
110	14:30,1.0271,5669.4508,5766.9816

```

111 14:40,1.0906,5664.1948,5521.5066
112 14:50,0.9376,5672.7011,5268.781
113 15:00,0.6694,5682.2544,5009.7152
114 15:10,0.66,5669.0195,4743.4484
115 15:20,0.7217,5656.3651,4471.224
116 15:30,0.7308,5657.0867,4188.2025
117 15:40,0.7534,5662.6008,3889.3931
118 15:50,0.7823,5669.2596,3594.5946
119 16:00,0.8055,5673.4767,3303.8878
120 16:10,0.8271,5671.0146,2998.4247
121 16:20,0.8519,5661.5926,2687.4131
122 16:30,0.8791,5665.8922,2383.1779
123 16:40,0.9024,5689.0372,2089.1246
124 16:50,0.9163,5603.6614,1791.0829
125 17:00,0.9264,5466.7124,1501.7283
126 17:10,0.9462,5467.3939,1255.1723
127 17:20,0.9742,5464.727,1036.3323
128 17:30,0.9862,5492.4458,836.7775
129 17:40,0.976,5621.7308,665.2313
130 17:50,1.0742,5091.2126,516.6758
131 18:00,1.2313,4208.5688,386.6731
132 18:10,1.2561,4122.7611,275.7933
133 18:20,1.2447,4255.9358,182.9526
134 18:30,1.2545,4225.9524,108.3414
135 18:40,1.2522,4233.3384,54.3121
136 18:50,1.2881,4246.4625,18.6569
137 19:00,1.3466,4239.1795,0.2753
138 19:10,1.3523,4244.6684,0.2296
139 19:20,1.3445,4251.9187,0.6007
140 19:30,1.3483,4256.6813,0
141 19:40,1.349,4267.3947,0
142 19:50,1.3473,4269.975,0
143 20:00,1.3482,4268.6702,0
144 20:10,1.3513,4282.2245,0
145 20:20,1.34,4291.7049,0
146 20:30,1.3489,4291.3611,0
147 20:40,1.3952,4293.9182,0
148 20:50,1.2226,4295.2135,0
149 21:00,0.9332,4302.0307,0
150 21:10,0.9068,4320.9618,0
151 21:20,0.9371,4302.3292,0
152 21:30,0.9371,4322.6758,0
153 21:40,0.9871,4428.8083,0
154 21:50,0.7764,4030.1614,0
155 22:00,0.4209,3362.7419,0
156 22:10,0.3859,3309.3934,0
157 22:20,0.4368,3412.9563,0
158 22:30,0.4198,3381.9313,0
159 22:40,0.4186,3386.5592,0
160 22:50,0.4241,3400.4846,0
161 23:00,0.4237,3399.2426,0
162 23:10,0.4248,3406.9809,0
163 23:20,0.424,3416.1518,0
164 23:30,0.422,3421.9811,0
165 23:40,0.424,3429.6923,0
166 23:50,0.4271,3439.061,0
167 0:00+1,0.4276,3444.7259,0""
168 T, DT = 144, 1 / 6 # 一天144个10分钟区间, 功率转电量乘DT。
169 PMAX, PMIN, EMIN, EMAX, EINIT = 5000., 1., 1200., 10800., 6000.
170 EPS_C, EPS_S = 1e-4, 1e-7 # 费用和无量纲第二目标的绝对容差。
171
172
173 def read_data(path=None, fallback=False):
174     """只读输入; 任何缺失、负数、时间错位都显式报错。"""
175     try:
176         if path is None:
177             rows = list(csv.reader(io.StringIO(EMBEDDED_CSV.strip())))[1:]
178         elif Path(path).suffix.lower() == ".xlsx":
179             from openpyxl import load_workbook # 仅外部XLSX输入需要该可选库。
180             book = load_workbook(path, data_only=True, read_only=True)

```

```

181         rows = list(book.active.values)[1:] # 跳过标题; 不会保存或修改源文件。
182         book.close()
183     else:
184         rows = None
185         for encoding in ("utf-8-sig", "gb18030"): # 兼容常用中文CSV编码。
186             try:
187                 with open(path, encoding=encoding, newline="") as handle:
188                     rows = list(csv.reader(handle))[1:]
189                     break
190             except UnicodeDecodeError:
191                 continue
192         if rows is None:
193             raise ValueError("CSV编码无法识别")
194         if len(rows) != T:
195             raise ValueError(f"应有144行数据, 实际{len(rows)}行")
196         for i, row in enumerate(rows): # 检查右端时刻, 避免错位十分钟。
197             label = str(row[0]).strip()
198             if i == T - 1 and label in ("0:00+1", "00:00+1", "24:00", "24:00:00", "00:00:00"):
199                 continue
200             pieces = label.split(":")
201             if len(pieces) < 2 or int(pieces[0]) * 60 + int(pieces[1]) != (i + 1) * 10:
202                 raise ValueError(f"第{i+1}行时间不连续: {label}")
203         data = np.asarray([[float(x) for x in row[1:4]] for row in rows])
204         if data.shape != (T, 3) or not np.isfinite(data).all():
205             raise ValueError("存在空值、非数值或非有限数")
206         if (data < 0).any() or (data[:, 0] <= 0).any():
207             raise ValueError("本模型要求正电价以及非负负载、光伏")
208         return data
209     except Exception as exc: # 同时兼容Excel压缩包损坏、编码和格式解析异常。
210         if path is not None and fallback:
211             print(f"警告: 读取失败({exc}), 明确回退到内嵌附件1", file=sys.stderr)
212             return read_data()
213         raise ValueError(f"输入读取失败: {exc}; 可用--fallback允许内嵌回退") from exc
214
215
216 def solve(data, delta=.001, ramp=1000., up=3, down=2, eta_rt=.9):
217     """同一可行域内依次求费用、平稳性、购电量; 全部层均保留整数互斥。"""
218     if not (0 < eta_rt <= 1 and delta >= 0 and 1 <= up <= T and 1 <= down <= T):
219         raise ValueError("效率、让步比例或持续时间不合法")
220     eta = np.sqrt(eta_rt) # 本问保持往返效率口径。
221     names = ("g", "c", "d", "w", "E", "zc", "zd", "sc", "sd", "oc", "od", "v",
222             "bc1", "bc2", "bd1", "bd2") # b为旧基准短段缺额线性化变量。
223     ids, offset = {}, 0
224     for name in names:
225         size = T + 1 if name == "E" else T # 储电量包含145个边界。
226         ids[name] = np.arange(offset, offset + size)
227         offset += size
228     lo, hi, integer = np.zeros(offset), np.full(offset, np.inf), np.zeros(offset)
229     for name in ("c", "d"):
230         hi[ids[name]] = PMAx
231     lo[ids["E"]], hi[ids["E"]] = EMIN, EMAX
232     lo[ids["E"]][0, -1]] = EINIT
233     hi[ids["E"]][0, -1]] = EINIT # 日初日末均6000kWh。
234     for name in ("zc", "zd", "sc", "sd", "oc", "od", "bc1", "bc2", "bd1", "bd2"):
235         hi[ids[name]] = 1
236     integer[ids["zc"]], integer[ids["zd"]] = 1, 1 # 仅模式变量必须声明整数。
237     ri, ci, av, lower, upper = [], [], [], [], []
238     def add(items, lb=-np.inf, ub=np.inf):
239         """将一行稀疏线性约束加入模型。"""
240         row = len(lower)
241         for col, val in items:
242             ri.append(row); ci.append(int(col)); av.append(float(val))
243         lower.append(float(lb)); upper.append(float(ub))
244     for t in range(T):
245         prev = (t - 1) % T # 循环日跨午夜连接。
246         ix = lambda key, j=t: int(ids[key][j])
247         net = [(ix("c"), 1), (ix("d"), -1), (ix("c", prev), -1), (ix("d", prev), 1)]
248         rhs = (data[t, 1] - data[t, 2]) * DT
249         add([(ix("g"), 1), (ix("d"), DT), (ix("c"), -DT), (ix("w"), -1)], rhs, rhs)
250         add([(ix("E", t+1), 1), (ix("E"), -1), (ix("c"), -eta*DT), (ix("d"), DT/eta)], 0, 0)

```

```

251 add([(ix("zc"), 1), (ix("zd"), 1)], ub=1) # 禁止同时充放电。
252 add(net, -ramp, ramp) # 运行功率变化上限, 基准可设为10000。
253 add([(ix("v"), 1)] + [(k, -v) for k, v in net], lb=0)
254 add([(ix("v"), 1)] + net, lb=0) # v不小于净功率差的绝对值。
255 for mode in ("c", "d"):
256     z, oldz, start, stop = ix("z"+mode), ix("z"+mode, prev), ix("s"+mode), ix("o"+mode)
257     add([(ix(mode), 1), (z, -PMAX)], ub=0)
258     add([(ix(mode), 1), (z, -PMIN)], lb=0)
259     add([(start, 1), (z, -1), (oldz, 1)], lb=0)
260     add([(start, 1), (z, -1)], ub=0)
261     add([(start, 1), (oldz, 1)], ub=1)
262     add([(stop, 1), (oldz, -1), (z, 1)], lb=0)
263     add([(stop, 1), (oldz, -1)], ub=0)
264     add([(stop, 1), (z, 1)], ub=1) # s、o精确表示0-1转换。
265     add([(ix("z"+mode, (t+j)%T), 1) for j in range(up)] + [(start, -up)], lb=0)
266     add([(ix("z"+mode, (t+j)%T), 1) for j in range(down)] + [(stop, down)], ub=down)
267     for k in (1, 2): # 30分钟参考长度的两级短段缺额。
268         b = ix("b"+mode+str(k))
269         add([(b, 1), (start, -1)], ub=0)
270         for j in range(1, k+1):
271             add([(b, 1), (start, -1), (ix("z"+mode, (t+j)%T), 1)], lb=0)
272             add([(b, 1)] + [(ix("z"+mode, (t+j)%T), 1) for j in range(1, k+1)], ub=k)
273 cost, smooth, quantity = np.zeros(offset), np.zeros(offset), np.zeros(offset)
274 cost[ids["g"]], quantity[ids["g"]] = data[:, 0], 1
275 smooth[ids["v"]] = 1 / (2 * PMAX * T)
276 for name in ("sc", "sd"):
277     smooth[ids[name]] = 1 / (2 * T)
278 for name in ("bc1", "bc2", "bd1", "bd2"):
279     smooth[ids[name]] = 1 / (4 * T)
280 records, solutions = [], []
281 for stage, objective in enumerate((cost, smooth * 1e6, quantity), 1):
282     matrix = coo_matrix((av, (ri, ci)), shape=(len(lower), offset)).tocsc()
283     began = time.perf_counter()
284     result = milp(objective, integrality=integer, bounds=Bounds(lo, hi),
285                  constraints=LinearConstraint(matrix, lower, upper),
286                  options={"mip_rel_gap": 1e-9, "time_limit": 180.})
287     if not result.success or result.x is None:
288         raise RuntimeError(f"第{stage}层未证最优: {result.message}")
289     x = result.x # 不对连续解取整, 不静默接受超时解。
290     rec = dict(stage=stage, cost=float(cost@x), quantity=float(quantity@x),
291              smooth=float(smooth@x), gap=float(result.mip_gap),
292              seconds=time.perf_counter()-began, nodes=int(result.mip_node_count))
293     records.append(rec); solutions.append({k: x[v] for k, v in ids.items()})
294     if stage == 1:
295         budget = (1 + delta) * rec["cost"] + EPS_C
296         add(list(zip(ids["g"], data[:, 0])), ub=budget)
297     elif stage == 2:
298         add([(i, v*1e6) for i, v in enumerate(smooth) if v], ub=(rec["smooth"]+EPS_S)*1e6)
299 config = dict(delta=delta, ramp=ramp, up=up, down=down, eta_rt=eta_rt)
300 result = dict(config=config, stages=records, trajectory=solutions[-1], budget=budget)
301 result["metrics"] = audit(data, result) # 从物理量独立复算, 不只看求解器成功标志。
302 return result
303
304
305 def lengths(z, value=1):
306     """读取循环0-1序列的段长, 合并跨午夜的同模式段。"""
307     starts = [i for i in range(T) if z[i] == value and z[(i-1)%T] != value]
308     if not starts:
309         return [T] if np.all(z == value) else []
310     return [next(k for k in range(1, T+1) if z[(i+k)%T] != value) for i in starts]
311
312
313 def audit(data, result):
314     """独立核验供需、SOC、功率、互斥、段长、费用预算及层次目标。"""
315     q, cfg = result["trajectory"], result["config"]
316     g, c, d, w, E = (q[k] for k in ("g", "c", "d", "w", "E"))
317     eta = np.sqrt(cfg["eta_rt"])
318     net = c-d
319     on, off = [], []
320     for key in ("zc", "zd"):

```

```

321         z = np.rint(q[key]).astype(int) # 仅用于诊断整数变量，不更改求解值。
322         on.extend(lengths(z)); off.extend(lengths(z, 0))
323         balance = g + data[:, 2]*DT + d*DT - data[:, 1]*DT - c*DT - w
324         transition = np.diff(E)-eta*c*DT+d*DT/eta
325         tv = float(np.abs(net-np.roll(net, 1)).sum())
326         short = sum(max(0, 3-k) for k in on)
327         smooth = tv/(2*PMAX*T)+len(on)/(2*T)+short/(4*T)
328         metrics = dict(cost=float(data[:, 0][g]), quantity=float(g.sum()), tv=tv, starts=len(on),
329                        short_deficit=short, short_runs=sum(k<3 for k in on),
330                        min_on=min(on, default=T)*10, min_off=min(off, default=T)*10,
331                        max_step=float(np.abs(net-np.roll(net, 1)).max()),
332                        charge=float(c.sum()*DT), discharge=float(d.sum()*DT),
333                        surplus=float(w.sum()), emin=float(E.min()), emax=float(E.max()),
334                        balance=float(np.abs(balance).max()), state=float(np.abs(transition).max()),
335                        endpoint=float(max(abs(E[0]-EINIT), abs(E[-1]-EINIT))),
336                        simultaneous=float(np.minimum(c, d).max()), smooth=smooth,
337                        max_c=float(c.max()), max_d=float(d.max()),
338                        low=int(np.sum((np.abs(net)>1e-5)&(np.abs(net)<50))))
339         metrics["daily_balance"] = float(abs(g.sum() - (data[:, 1]-data[:, 2]).sum()*DT
340                                           - (c.sum()-d.sum()*DT - w.sum()))
341         metrics["cost_recompute"] = abs(metrics["cost"]-result["stages"][-1]["cost"])
342         tolerance = 2e-5 # kW/kWh诊断容差，目标函数另用各自量尺核查。
343         violations = [
344             metrics["balance"]>tolerance, metrics["state"]>tolerance,
345             metrics["endpoint"]>tolerance, metrics["simultaneous"]>tolerance,
346             E.min()<EMIN-tolerance, E.max()>EMAX+tolerance,
347             c.max()>PMAX+tolerance, d.max()>PMAX+tolerance,
348             min(g.min(), c.min(), d.min(), w.min()) < -tolerance,
349             metrics["max_step"]>cfg["ramp"]+tolerance,
350             metrics["min_on"]<cfg["up"]*10, metrics["min_off"]<cfg["down"]*10,
351             metrics["cost"]>result["budget"]+1e-5,
352             smooth>result["stages"][1]["smooth"]+EPS_S+1e-9,
353             np.max(np.abs(q["zc"]-np.rint(q["zc"])))>1e-6,
354             np.max(np.abs(q["zd"]-np.rint(q["zd"])))>1e-6,
355             np.max(q["zc"]+q["zd"])>1+1e-6,
356             np.max(PMIN*q["zc"]-c)>tolerance, np.max(c-PMAX*q["zc"]>tolerance,
357             np.max(PMIN*q["zd"]-d)>tolerance, np.max(d-PMAX*q["zd"]>tolerance,
358             metrics["daily_balance"]>tolerance, metrics["cost_recompute"]>1e-5,
359         ]
360         metrics["violations"] = sum(bool(v) for v in violations)
361         if metrics["violations"]:
362             raise RuntimeError(f"独立约束检验失败: {metrics}")
363         return metrics
364
365
366 def save_case(out, name, data, result):
367     """导出144段完整计划及每层状态，不覆盖用户原始表。"""
368     def serial(value):
369         if isinstance(value, np.ndarray):
370             return value.tolist()
371         if isinstance(value, np.generic):
372             return value.item()
373         raise TypeError(f"无法序列化: {type(value)}")
374     (out/(name+".json")).write_text(json.dumps(result, ensure_ascii=False, indent=2,
375                                                default=serial), encoding="utf-8")
376     q = result["trajectory"]
377     with (out/(name+".csv")).open("w", newline="", encoding="utf-8-sig") as handle:
378         writer = csv.writer(handle)
379         writer.writerow(["起点/h", "终点/h", "电价/元每kWh", "负载/kW", "光伏/kW",
380                        "购电量/kWh", "充电功率/kW", "放电功率/kW", "剩余/kWh",
381                        "期初储电/kWh", "期末储电/kWh", "区间电费/元"])
382         for t in range(T):
383             writer.writerow([t*DT, (t+1)*DT, *data[t], q["g"][t], q["c"][t], q["d"][t],
384                             q["w"][t], q["E"][t], q["E"][t+1], data[t, 0]*q["g"][t]])
385
386
387 def draw(out, data, cases, sweep):
388     """依据真实解绘制阶梯功率、储电轨迹和费用-总变差图。"""
389     os.environ.setdefault("MPLCONFIGDIR", str((out/"mpl-cache").resolve()))
390     import matplotlib

```

```

391 matplotlib.use("Agg") # 无图形界面的机器也能输出。
392 import matplotlib.pyplot as plt
393 plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
394                     "axes.unicode_minus": False, "font.size": 10,
395                     "axes.spines.top": False, "axes.spines.right": False})
396 edges = np.arange(T+1)*DT
397 fig, axes = plt.subplots(2, 1, figsize=(10, 6.4), sharex=True, layout="constrained")
398 for key, label, color in [("baseline", "严格经济基准", "#8061A8"), ("revised", "平稳运行方案", "#138B99")]:
399     q = cases[key]["trajectory"]
400     axes[0].stairs(q["c"]-q["d"], edges, label=label, color=color, linewidth=1.6)
401     axes[1].plot(edges, q["E"], label=label, color=color, linewidth=1.6)
402 axes[0].axhline(0, color="#777777", linewidth=.6)
403 axes[0].set_ylabel("净充电功率 / kW") # 标题与图例分别留出空间。
404 axes[0].set_title("典型日电池净功率与储电量", pad=34)
405 axes[1].axhline(EMIN, color="#B27838", linestyle="--", label="安全储电边界")
406 axes[1].axhline(EMAX, color="#B27838", linestyle="--")
407 axes[1].set(xlabel="时刻 / h", ylabel="储电量 / kWh", xlim=(0,24), xticks=np.arange(0,25,2))
408 for ax in axes:
409     ax.legend(ncol=3, fontsize=9, loc="lower center",
410             bbox_to_anchor=(.5, 1.01), frameon=False) # 图例不遮挡数据轨迹。
411     ax.grid(alpha=.16)
412 fig.savefig(out/"q1_dispatch.png", dpi=320); plt.close(fig)
413 rows = sorted(sweep, key=lambda row: row["delta"])
414 fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), layout="constrained")
415 x = [row["delta"]*100 for row in rows]
416 for ax, key, ylabel, color in [(axes[0], "cost", "购电费 / 元", "#B27838"),
417                               (axes[1], "tv", "循环功率总变差 / kW", "#138B99")]:
418     ax.plot(x, [row[key] for row in rows], "o-", color=color, label="单因素重求解")
419     ax.axvline(.1, color="#8061A8", linestyle="--", label="选定让步比例")
420     ax.set(xlabel="电费让步比例 / %", ylabel=ylabel); ax.grid(alpha=.16); ax.legend(fontsize=8)
421 fig.suptitle("经济让步比例对电费与运行平稳性的影响")
422 fig.savefig(out/"q1_tradeoff.png", dpi=320); plt.close(fig)
423
424
425 def main():
426     parser = argparse.ArgumentParser(description=__doc__)
427     parser.add_argument("--input", help="可选CSV或XLSX; 省略则读取内嵌附件1")
428     parser.add_argument("--fallback", action="store_true", help="外部输入失败时显式回退")
429     parser.add_argument("--out", default="q1_output", help="生成结果目录")
430     parser.add_argument("--quick", action="store_true", help="只做主方案和严格经济基准")
431     args = parser.parse_args()
432     data = read_data(args.input, args.fallback)
433     out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
434     cases, sweep, sensitivity = {}, [], []
435     tasks = [("revised", {}), ("baseline", dict(delta=0, ramp=10000, up=1, down=1))]
436     if not args.quick:
437         tasks += [(f"delta_{v}", dict(delta=v)) for v in (0, .0005, .002, .005)]
438         tasks += [(f"{key}_{factor}", {key: value*factor}) for key, value in
439                 [("delta", .001), ("ramp", 1000.), ("eta_rt", .9)] for factor in (.9, 1.1)]
440     for name, settings in tasks:
441         print(f"求解 {name}", flush=True)
442         result = solve(data, **settings)
443         cases[name] = result; save_case(out, name, data, result)
444         row = dict(name=name, **result["config"], **result["metrics"])
445         sensitivity.append(row)
446         if name == "revised" or name.startswith("delta_") and name not in ("delta_0.9", "delta_1.1"):
447             :
448             sweep.append(row)
449         print(json.dumps(row, ensure_ascii=False), flush=True)
450     noise = []
451     if not args.quick:
452         for seed in range(2026, 2029): # 三次局部压力测试, 不宣称统计置信保证。
453             rng = np.random.default_rng(seed)
454             perturbed = data.copy()
455             factors = 1 + np.clip(rng.normal(0, .01, (T,2)), -.03, .03)
456             perturbed[:, 1:] *= factors # 夜间零光伏保留为零。
457             result = solve(perturbed)
458             save_case(out, f"noise_{seed}", perturbed, result)
459             fixed = cases["revised"]["trajectory"]

```

```

459     gap = ((perturbed[:, 1]-perturbed[:, 2])-(data[:, 1]-data[:, 2]))*DT-fixed["w"]
460     row = dict(seed=seed, cost=result["metrics"]["cost"],
461               change_pct=(result["metrics"]["cost"]/cases["revised"]["metrics"]["cost"]-1)
462               *100,
463               fixed_plan_deficit_kwh=float(np.maximum(gap,0).sum()),
464               violations=result["metrics"]["violations"])
465     noise.append(row); print("噪声检验", row, flush=True)
466     final = cases["revised"]["trajectory"] # 为题目指定时段另外输出便于核对的汇总。
467     specified = [{"start_hour": t/6, "purchase_kwh": float(final["g"][t])}
468                 for t in (60, 72, 84, 96, 108, 120)]
469     blocks = [{"start_hour": 4*j, "end_hour": 4*(j+1),
470               "charge_kwh": float(final["c"][24*j:24*(j+1)].sum()*DT),
471               "discharge_kwh": float(final["d"][24*j:24*(j+1)].sum()*DT)} for j in range(6)]
472     summary = dict(environment=dict(python=sys.version, numpy=np.__version__, scipy=scipy.
473                                   __version__),
474                    no_storage_cost=float(data[:,0]@np.maximum(data[:,1]-data[:,2],0)*DT),
475                    load_kwh=float(data[:,1].sum()*DT), pv_kwh=float(data[:,2].sum()*DT),
476                    cases=sensitivity, noise=noise, specified=specified, storage_blocks=blocks,
477                    initial_kwh=float(final["E"][0]), final_kwh=float(final["E"][-1]))
478     (out/"summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf
479     -8")
480     if len(sweep)>1:
481         draw(out, data, cases, sweep)
482     print("完成; 144段计划、各层求解状态及检验结果位于", out.resolve(), flush=True)
483
484 if __name__ == "__main__":
485     try:
486         main()
487     except (ValueError, RuntimeError, OSError, ImportError) as exc:
488         print(f"运行终止: {exc}", file=sys.stderr)
489         sys.exit(1) # 不输出虚构的成功结论。

```